

3D Smart Ambulance Routing System Using Graph Algorithms

Arsen Hamzaj Faculty of Computer Science and IT
Universiteti Metropolitan Tirana
arsen.hamzaj24@umt.edu.al

Bajram Haxhiu Faculty of Computer Science and IT
Universiteti Metropolitan Tirana
bajram.haxhiu24@umt.edu.al

Mateos Sharka Faculty of Computer Science and IT
Universiteti Metropolitan Tirana
mateos.sharka24@umt.edu.al

May 18, 2026

Abstract

Emergency response routing is a critical real-world optimization problem where fast decision-making can significantly affect patient survival. This paper presents a 3D Smart Ambulance Routing System developed using Java and JavaFX 3D. The city is modeled as a weighted graph, where hospitals, intersections, and patient locations are represented as vertices, while roads are represented as weighted edges.

The system implements Breadth-First Search, Depth-First Search, Dijkstra's Algorithm, and A* Search. These algorithms are used to analyze traversal behavior, shortest-path discovery, and emergency route optimization. The project also includes multiple ambulances, Albanian hospital nodes, weighted road visualization, day/night mode, sound effects, and real-time 3D ambulance movement.

The main goal of this project is to demonstrate how graph algorithms and data structures can be applied to an emergency dispatch system. The results show that uninformed algorithms such as BFS and DFS are useful for exploration, while Dijkstra and A* are more suitable for weighted route optimization.

Graph Algorithms, Dijkstra Algorithm, A* Search, JavaFX 3D, Emergency Routing, Weighted Graphs, Ambulance Dispatch System, Data Structures and Algorithms.

1 Introduction

Graph algorithms are widely used in navigation systems, emergency response systems, transportation planning, robotics, artificial intelligence, and communication networks. Any environment consisting of connected locations can be represented as a graph. In such a model, locations are vertices and the connections between them are edges.

In emergency medical systems, route optimization is especially important. An ambulance must reach the patient as quickly as possible and then return to the hospital using the most efficient route. A poor routing decision may increase response time and reduce the effectiveness of emergency care.

This project, titled *3D Smart Ambulance Routing System Using Graph Algorithms*, models a city as a weighted graph and applies graph algorithms to determine optimal ambulance routes. The system also provides a 3D visualization layer using JavaFX 3D, allowing the user to observe the graph, roads, hospitals, ambulances, and selected paths.

2 Motivation

The motivation behind this project is to connect theoretical Data Structures and Algorithms concepts with a realistic emergency-routing scenario. Many DSA projects demonstrate algorithms only in console-based or simple graphical environments. This project improves the presentation by combining algorithmic logic with a 3D simulation.

The system demonstrates how graph theory can solve practical problems such as:

- selecting the nearest ambulance,
- calculating shortest routes,
- comparing pathfinding algorithms,
- visualizing weighted roads,
- simulating emergency movement in real time.

3 Problem Formulation

The city network is modeled as a weighted graph:

$$G = (V, E, w) \tag{1}$$

where:

- V is the set of vertices,

- E is the set of edges,
- w is the positive weight function assigned to each edge.

Each vertex represents a hospital, intersection, or patient location. Each edge represents a road between two locations. The edge weight represents distance, travel time, or traffic cost.

A path from source s to destination t is defined as:

$$P = \langle v_0, v_1, v_2, \dots, v_k \rangle \quad (2)$$

where:

$$v_0 = s, \quad v_k = t, \quad (v_i, v_{i+1}) \in E \quad (3)$$

The total path cost is:

$$cost(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}) \quad (4)$$

The shortest path is therefore:

$$P^* = \arg \min_{P \in P(s,t)} cost(P) \quad (5)$$

4 System Design

The project is designed using a modular architecture. The purpose of this architecture is to separate graph logic, algorithm implementation, visualization, and user interface components.

4.1 Main System Modules

Table 1: Main System Modules

Module	Responsibility
Graph Module	Stores nodes, edges, and weights
Algorithm Module	Implements BFS, DFS, Dijkstra, and A*
Ambulance Module	Controls ambulance movement
Visualization Module	Renders JavaFX 3D environment
Sound Module	Handles siren and notification sounds
Dashboard Module	Displays route and algorithm statistics

4.2 Data Flow Diagram

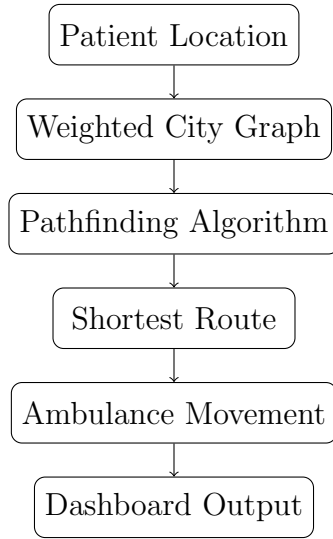


Figure 1: Data flow of the ambulance routing system

5 Graph Representation

The project uses an adjacency list representation. This is more efficient than an adjacency matrix for sparse graphs, which are common in road networks.

For every node u , the adjacency list stores all connected neighbors:

$$Adj(u) = \{(v, w(u, v)) \mid (u, v) \in E\} \quad (6)$$

The memory complexity of the adjacency list is:

$$O(|V| + |E|) \quad (7)$$

5.1 Node Types

Table 2: Graph Node Types

Node Type	Description
Hospital	Emergency center with ambulance
Intersection	Road crossing point
Patient	Emergency destination
Road Node	Intermediate route point

5.2 Edge Weights

Road weights are assigned values from 1 to 20.

Table 3: Road Weight Interpretation

Weight Range	Meaning
1–5	Fast road
6–10	Normal traffic
11–15	Heavy traffic
16–20	Severe congestion

6 Algorithms Implemented

6.1 Breadth-First Search

Breadth-First Search explores the graph level by level using a queue. It is useful for unweighted graphs and traversal visualization.

Algorithm 1 Breadth-First Search

Require: Graph G , source s

```
1: Create empty queue  $Q$ 
2: Mark  $s$  as visited
3: Enqueue  $s$ 
4: while  $Q$  is not empty do
5:    $u \leftarrow Q.dequeue()$ 
6:   for each neighbor  $v$  of  $u$  do
7:     if  $v$  is not visited then
8:       Mark  $v$  as visited
9:       Enqueue  $v$ 
10:    end if
11:  end for
12: end while
```

Time complexity:

$$O(V + E) \tag{8}$$

6.2 Depth-First Search

Depth-First Search explores deeply along one branch before backtracking. It is useful for graph exploration but does not guarantee the shortest path.

Algorithm 2 Depth-First Search

Require: Graph G , source s

```
1: Mark  $s$  as visited
2: for each neighbor  $v$  of  $s$  do
3:   if  $v$  is not visited then
4:     DFS( $v$ )
5:   end if
6: end for
```

Time complexity:

$$O(V + E) \tag{9}$$

6.3 Dijkstra's Algorithm

Dijkstra's Algorithm finds the shortest path in weighted graphs with non-negative edge weights. It uses a priority queue ordered by the shortest known distance.

For every vertex v :

$$dist[v] = \text{minimum known distance from source to } v \tag{10}$$

Relaxation step:

$$dist[v] = \min(dist[v], dist[u] + w(u, v)) \tag{11}$$

Algorithm 3 Dijkstra Algorithm

Require: Weighted graph G , source s , target t

```
1: Set  $dist[s] = 0$ 
2: Set all other distances to infinity
3: Insert  $s$  into priority queue
4: while priority queue is not empty do
5:    $u \leftarrow$  node with minimum distance
6:   if  $u = t$  then
7:     return shortest path
8:   end if
9:   for each edge  $(u, v)$  do
10:     $newDist \leftarrow dist[u] + w(u, v)$ 
11:    if  $newDist < dist[v]$  then
12:       $dist[v] \leftarrow newDist$ 
13:       $parent[v] \leftarrow u$ 
14:      Insert  $v$  into priority queue
15:    end if
16:  end for
17: end while
```

Time complexity:

$$O((V + E) \log V) \quad (12)$$

6.4 A* Search

A* Search improves Dijkstra by using a heuristic estimate toward the destination.

$$f(n) = g(n) + h(n) \quad (13)$$

where:

- $g(n)$ is the exact cost from the source to node n ,
- $h(n)$ is the estimated cost from node n to the target,
- $f(n)$ is the total estimated cost.

For coordinate-based graphs, the Euclidean heuristic is:

$$h(n) = \sqrt{(x_n - x_t)^2 + (y_n - y_t)^2} \quad (14)$$

Algorithm 4 A* Search

Require: Graph G , source s , target t

```
1: Set  $g[s] = 0$ 
2: Insert  $s$  into priority queue with priority  $f(s)$ 
3: while priority queue is not empty do
4:    $u \leftarrow$  node with lowest  $f(u)$ 
5:   if  $u = t$  then
6:     return reconstructed path
7:   end if
8:   for each neighbor  $v$  of  $u$  do
9:      $newG \leftarrow g[u] + w(u, v)$ 
10:    if  $newG < g[v]$  then
11:       $g[v] \leftarrow newG$ 
12:       $f[v] \leftarrow g[v] + h(v)$ 
13:       $parent[v] \leftarrow u$ 
14:    end if
15:  end for
16: end while
```

Time complexity:

$$O((V + E) \log V) \quad (15)$$

7 Ambulance Dispatch System

The project includes multiple ambulances assigned to hospitals. Each ambulance has a current graph position. When a patient appears, the system calculates the shortest path from each ambulance to the patient.

The selected ambulance is:

$$A^* = \arg \min_{A_i} d(A_i, P) \quad (16)$$

where:

- A_i is ambulance i ,
- P is the patient node,
- $d(A_i, P)$ is the shortest path cost.

7.1 Hospitals Used

Table 4: Hospitals in the Simulation

Hospital	Ambulance
QSUT Mother Teresa	Ambulance 1
American Hospital Tirana	Ambulance 2

8 3D Visualization

The application uses JavaFX 3D to create an interactive city simulation.

Table 5: JavaFX 3D Components

Object	JavaFX Representation
Buildings	Box
Intersections	Sphere
Roads	3D lines or cylinders
Ambulance	3D object / Box model
Hospitals	Highlighted buildings
Patient	Highlighted node

The visualization includes:

- weighted road labels,
- moving ambulances,
- highlighted shortest path,
- day and night mode,
- sound effects,
- emergency route simulation.

9 Day and Night Mode

The system includes a day/night toggle. Day mode uses bright lighting and clear road colors. Night mode uses darker materials, glowing roads, and emergency light effects.

This feature improves visual presentation and makes the simulation more realistic.

10 Sound System

The project includes sound effects using JavaFX media support. Sounds are activated during important simulation events.

Table 6: Sound Events

Sound	Trigger
Dispatch sound	Emergency generated
Route sound	Shortest path found
Ambulance siren	Ambulance starts moving

11 Experimental Evaluation

The algorithms are compared using:

- explored nodes,
- route cost,
- execution time,
- weighted graph suitability.

Table 7: Algorithm Comparison

Algorithm	Explored Nodes	Path Cost	Time
BFS	15	Not optimal	Fast
DFS	22	Not optimal	Medium
Dijkstra	9	Optimal	Efficient
A*	6	Optimal	Very efficient

11.1 Explored Nodes Chart

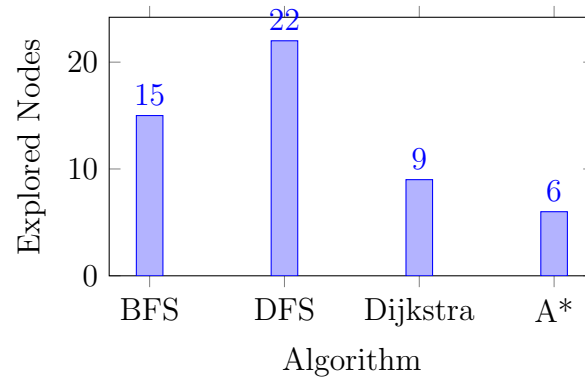


Figure 2: Comparison of explored nodes

11.2 Path Cost Chart

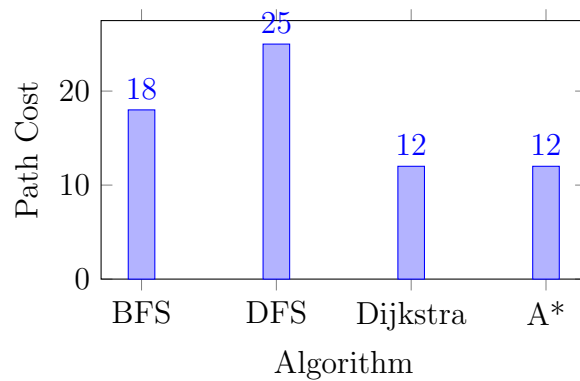


Figure 3: Comparison of route cost

12 Discussion

The results confirm that each graph algorithm has a different role in the routing system. BFS is useful for simple traversal and unweighted paths, but it does not consider edge weights. DFS is useful for exploration but may return long or inefficient paths.

Dijkstra is more suitable for ambulance routing because it guarantees the shortest path in weighted graphs with non-negative edge weights. However, it may explore more nodes than necessary because it does not know the direction of the target.

A* improves performance by using a heuristic function. Since the graph nodes have coordinates, A* can estimate the distance to the patient and guide the search more efficiently. This makes A* particularly useful for real-time emergency route optimization.

The 3D visualization layer improves the educational value of the project because users can observe how algorithms behave visually. Instead of only seeing numerical output,

users can see the ambulance moving along the selected route.

13 Limitations

Although the system is functional and realistic for a university project, it has some limitations:

- traffic values are simulated,
- no real GPS data is used,
- the city graph is limited in size,
- ambulance physics are simplified,
- road weights are manually assigned.

14 Future Work

Future improvements may include:

- real map integration,
- live GPS tracking,
- real-time traffic APIs,
- larger city graphs,
- machine learning for traffic prediction,
- more realistic ambulance models,
- hospital capacity management.

15 Conclusion

This paper presented a 3D Smart Ambulance Routing System based on graph algorithms. The city was modeled as a weighted graph, and several algorithms were implemented to analyze traversal and shortest-path behavior.

The project demonstrates that graph algorithms are highly effective for emergency route optimization. BFS and DFS provide useful traversal behavior, while Dijkstra and A* are more suitable for weighted shortest-path problems.

The integration of JavaFX 3D visualization, multiple ambulances, weighted roads, Albanian hospital nodes, day/night mode, and sound effects makes the project professional and presentation-ready.

Overall, the system successfully combines Data Structures and Algorithms theory with real-world emergency dispatch simulation.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press.
- [2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, 1959.
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, 1968.
- [4] Oracle, “JavaFX Documentation,” Oracle Developer Documentation.
- [5] R. Sedgewick and K. Wayne, *Algorithms*. Addison-Wesley.